

Chapter 8 Summary

Algebraic Hierarchies in Lean 4

Reading notes

Abstract

Lean’s mathematical library organizes algebraic structures (groups, rings, modules) and the maps between them (homomorphisms) as a layered hierarchy of **class/structure** declarations. Each layer **extends** the layer(s) below it and adds *exactly one* new piece of data or axiom, so no property is ever proved twice. This note walks through the full hierarchy built in Chapter 8: from the bare **Dia/One/Inv** building blocks, up through **Monoid** and **Group**, into **Ring** and **Module**, and finally the parallel hierarchy of structure-preserving morphisms. A complete reference table is given in Section 7.

Contents

1	The Abstract Group Hierarchy	1
1.1	The raw ingredients: Dia , One , Inv	1
1.2	Adding laws: Semigroup and DiaOneClass	2
1.3	Monoid and Group	2
2	Specializing the Operator: + and *	2
2.1	Building the additive side directly	3
2.2	The to_additive attribute	3
2.3	Commutative variants and Additive Commutative Group	3
3	Ring: Two Operators on One Carrier	4
3.1	Commutativity of + is derived, not assumed	4
4	Module: Two Carrier Sets, One Scalar Action	5
4.1	A ring is a module over itself	5
5	The export Mechanism	6
6	Morphisms: Structure-Preserving Maps	6
6.1	Monoid , additive monoid, and ring homomorphisms	6
6.2	Homomorphism classes	6
6.3	Order-preserving homomorphisms	7
7	Full Hierarchy Reference	8

1 The Abstract Group Hierarchy

A *group* has one carrier type, one binary operator (abstracted as $\diamond$), and one identity element (abstracted as **1**). Lean builds it up from three independent, minimal pieces — **Dia**, **One**, **Inv** — and then combines them.

1.1 The raw ingredients: Dia, One, Inv

Dia₁ --- the operator

```
class Dia1 (a : Type) where
  dia : a -> a -> a
  -- notation:  a diamond b  ~~>  a <> b
```

Declares a binary operation $\diamond : \alpha \rightarrow \alpha \rightarrow \alpha$ with no properties attached yet — not even associativity.

One₁ --- the identity

```
class One1 (a : Type) where
  one : a
  -- notation:  One1.one  ~~>  1 (as a distinguished element, not the
    literal 1)
```

Only asserts that *some* distinguished element **1** exists in α ; it says nothing about how it interacts with $\diamond$.

Inv₁ --- the inverse function

```
class Inv1 (a : Type) where
  inv : a -> a
  -- notation:  Inv1.inv a  ~~>  a^{-1}
```

A unary function $\alpha \rightarrow \alpha$, again with no law relating it to $\diamond$ or **1** yet.

1.2 Adding laws: Semigroup and DiaOneClass

Semigroup₁ --- extends Dia₁

```
class Semigroup1 (a : Type) extends Dia1 a where
  dia_assoc : forall x y z : a, (x <> y) <> z = x <> (y <> z)
```

Adds associativity of $\diamond$. Nothing about identity yet.

DiaOneClass₁ --- extends One₁, Dia₁

```
class DiaOneClass1 (a : Type) extends One1 a, Dia1 a where
  one_dia : forall x : a, 1 <> x = x  -- one is a left neutral
    element
  dia_one : forall x : a, x <> 1 = x  -- one is a right neutral
    element
```

Adds the two neutral-element laws that relate **1** and $\diamond$. It needs *both* **One** and **Dia**, but not associativity.

1.3 Monoid and Group

Monoid₁ --- extends Semigroup₁, DiaOneClass₁

```
class Monoid1 (a : Type) extends Semigroup1 a, DiaOneClass1 a
```

Adds *no new axioms* — it is purely the conjunction of associativity (from **Semigroup**) and the two identity laws (from **DiaOneClass**). This is the pattern used repeatedly in this hierarchy: a

class that only combines two independent parents.

Group₁ --- extends Monoid₁, Inv₁

```
class Group1 (G : Type) extends Monoid1 G, Inv1 G where
  inv_dia : forall x : G, x-1 <> x = 1
```

Adds the single new law $x^{-1} \diamond x = 1$ on top of everything Monoid already provides, and requires Inv for the inverse function itself.

Key Idea

Every arrow in this hierarchy is an **extends** clause, and every class adds *at most one* new axiom relative to what it extends. Monoid = Semigroup $\wedge$ DiaOneClass (no new axiom); Group = Monoid $\wedge$ Inv, plus exactly one new law ($x^{-1} \diamond x = 1$).

2 Specializing the Operator: + and *

The abstract pair $(\diamond, 1)$ cannot be used directly in ordinary mathematics — we always work with a *concrete* operator. Instantiating with $(+, 0)$ gives the Add-prefixed hierarchy; instantiating with $(*, 1)$ gives the un-prefixed hierarchy, which is Lean’s default (i.e. plain Group means *multiplicative* group).

2.1 Building the additive side directly

AddSemigroup₃ --- extends Add

```
class AddSemigroup3 (a : Type) extends Add a where
  add_assoc3 : forall x y z : a, (x + y) + z = x + (y + z)
```

Add plays the role Dia played in the abstract hierarchy, specialized to +.

AddMonoid₃ --- extends AddSemigroup₃, AddZeroClass

```
class AddMonoid3 (a : Type) extends AddSemigroup3 a, AddZeroClass a
```

AddZeroClass plays the role of DiaOneClass for $(+, 0)$. As with Monoid, no new axiom is added here.

AddGroup₃ --- extends AddMonoid₃, Neg

```
class AddGroup3 (G : Type) extends AddMonoid3 G, Neg G where
  neg_add : forall x : G, -x + x = 0
```

Neg plays the role of Inv; the group law becomes $-x + x = 0$.

2.2 The to_additive attribute

Rather than writing the multiplicative and additive hierarchies twice, Lean writes the multiplicative version and tags it `@[to_additive AddX]`, which auto-generates the additive class AddX by textually replacing $* \rightarrow +$, $1 \rightarrow 0$, $\text{inv} \rightarrow \text{neg}$, etc.

```
@[to_additive AddSemigroup3]
class Semigroup3 (a : Type) extends Mul a where
  mul_assoc3 : forall x y z : a, (x * y) * z = x * (y * z)
```

```
@[to_additive AddGroup3]
class Group3 (G : Type) extends Monoid3 G, Inv G where
  inv_mul : forall x : G, x-1 * x = 1
```

Key Idea

Only *one* side of each pair is proved by hand; @[to_additive] generates the other automatically along with its lemmas and notation. This is why the source only proves the multiplicative Group3 in full, and gets AddGroup3 for free.

2.3 Commutative variants and Additive Commutative Group

```
class AddCommSemigroup3 (a : Type) extends AddSemigroup3 a where
  add_comm : forall x y : a, x + y = y + x

@[to_additive AddCommSemigroup3]
class CommSemigroup3 (a : Type) extends Semigroup3 a where
  mul_comm : forall x y : a, x * y = y * x

class AddCommMonoid3 (a : Type) extends AddMonoid3 a, AddCommSemigroup3 a

@[to_additive AddCommMonoid3]
class CommMonoid3 (a : Type) extends Monoid3 a, CommSemigroup3 a

class AddCommGroup3 (G : Type) extends AddGroup3 G, AddCommMonoid3 G
```

AddCommGroup (an additive group with commutativity) is the standard name for what mathematicians call an *Abelian group*. Because $*$ is Lean’s default operator, one sometimes also sees “Abelian group” used loosely to refer to the multiplicative CommGroup — the two are the same structure up to to_additive.

3 Ring: Two Operators on One Carrier

A *ring* R combines **both** operators on the *same* carrier set: an additive structure and a multiplicative structure, glued together by an absorption law and distributivity.

- object: AddGroup R with $+$ (*not* AddCommGroup — see below)
- scalar: Monoid R with $*$
- $0 * a = a * 0 = 0$ (MulZeroClass)
- $a * (b + c) = a * b + a * c$ and $(a + b) * c = a * c + b * c$ (left/right distributivity)

```
Ring3 --- extends AddGroup3, Monoid3, MulZeroClass
```

```
class Ring3 (R : Type) extends AddGroup3 R, Monoid3 R, MulZeroClass R
  where
    left_distrib : forall a b c : R, a * (b + c) = a * b + a * c
    right_distrib : forall a b c : R, (a + b) * c = a * c + b * c
```

In general a ring is *not* assumed to have multiplicative commutativity.

3.1 Commutativity of + is derived, not assumed

Ring3 extends the plain AddGroup3, not AddCommGroup3 — yet every ring’s addition *is* commutative. The reason: commutativity of + is a theorem that follows from distributivity, so it does not need to be assumed as an extra axiom.

$$(1 + 1) * (a + b) = (a + b) + (a + b) = (a + a) + (b + b)$$

Expanding the left-hand side two different ways (via `right_distrib` then `left_distrib`, or vice versa) and cancelling gives $a + b = b + a$. Lean proves this directly:

```
instance {R : Type} [Ring3 R] : AddCommGroup3 R :=
{ add_comm := by
  intro a b
  have : a + (a + b + b) = a + (b + a + b) := calc
    a + (a + b + b) = (a + a) + (b + b)           := by simp [add_assoc3]
  _ = (1 * a + 1 * a) + (1 * b + 1 * b)           := by simp
  _ = (1 + 1) * a + (1 + 1) * b                   := by simp [Ring3.
    right_distrib]
  _ = (1 + 1) * (a + b)                           := by simp [Ring3.
    left_distrib]
  _ = 1 * (a + b) + 1 * (a + b)                     := by simp [Ring3.
    right_distrib]
  _ = (a + b) + (a + b)                             := by simp
  _ = a + (b + a + b)                             := by simp [add_assoc3]
]
exact add_right_cancel3 (add_left_cancel3 this) }
```

Key Idea

Whenever a stronger property can be *proved* from weaker axioms already in scope, Lean’s hierarchy design avoids asserting it as an independent axiom. This keeps each class’s axiom list minimal and keeps instance proofs shorter (fewer obligations to discharge when constructing a concrete ring).

4 Module: Two Carrier Sets, One Scalar Action

A *module* again has two components — an object and a scalar — but this time they live on **two different** carrier sets, related by a scalar-multiplication action •.

- the object M is an AddCommGroup with operator +
- the scalar R is a Ring with operators * and +
- the action $\bullet : R \rightarrow M \rightarrow M$ (SMul, “scalar-mul”) connects them

SMul₃ and Module₁

```
class SMul3 (a : Type) (b : Type) where
  smul : a -> b -> b
  -- notation: a bullet m ~> a SMul3.smul m

class Module1 (R : Type) [Ring3 R] (M : Type) [AddCommGroup3 M]
  extends SMul3 R M where
  zero_smul : forall m : M, (0 : R) . m = 0
  one_smul  : forall m : M, (1 : R) . m = m
  mul_smul  : forall (a b : R) (m : M), (a * b) . m = a . b . m
```

```

add_smul  : forall (a b : R) (m : M), (a + b) . m = a . m + b . m
smul_add  : forall (a : R) (m n : M), a . (m + n) = a . m + a . n

```

Five axioms govern how scalars interact with M 's zero, R 's identity, R 's multiplication, and both distributivity directions.

4.1 A ring is a module over itself

Since `Ring` already has all the structure `Module` needs (an `AddCommGroup` to act on, and a `Ring` to act with), R can be viewed as a module over itself, with $\bullet$ instantiated to $*$:

```

instance selfModule (R : Type) [Ring3 R] : Module1 R R where
  smul      := fun r s => r * s
  zero_smul := zero_mul
  one_smul  := one_mul
  mul_smul  := mul_assoc3
  add_smul  := Ring3.right_distrib
  smul_add  := Ring3.left_distrib

```

Module field	Abstract statement	Instantiated as (selfModule)
<code>smul</code>	$a \bullet m$	$a * m$
<code>zero_smul</code>	$0_R \bullet m = 0_M$	<code>zero_mul</code>
<code>one_smul</code>	$1_R \bullet m = m$	<code>one_mul</code>
<code>mul_smul</code>	$(a * b) \bullet m = a \bullet (b \bullet m)$	<code>mul_assoc3</code>
<code>add_smul</code>	$(a + b) \bullet m = a \bullet m + b \bullet m$	<code>Ring3.right_distrib</code>
<code>smul_add</code>	$a \bullet (m + n) = a \bullet m + a \bullet n$	<code>Ring3.left_distrib</code>

Key Idea

`selfModule` is *not* a fully concrete instance — R is still an arbitrary `Ring`. It is a bridge lemma: once it exists, any concrete ring ($\mathbb{Z}$, $\mathbb{F}[X]$, matrix rings, ...) automatically gets a module-over-itself instance, and every generic module lemma becomes available for it for free.

5 The export Mechanism

Field names declared inside a `class` are normally accessed with a qualified name such as `DiaOneClass.one_dia`. The `export` command re-exposes them as unqualified top-level names:

```

export DiaOneClass1 (one_dia dia_one)
export Semigroup1 (dia_assoc)
export Group1 (inv_dia)

```

After this, `one_dia`, `dia_one`, `dia_assoc`, and `inv_dia` can be used directly, without the class-name prefix — purely a naming convenience, with no effect on the type hierarchy itself.

6 Morphisms: Structure-Preserving Maps

A homomorphism bundles a function together with *proofs* that it preserves every operation of the source structure.

6.1 Monoid, additive monoid, and ring homomorphisms

MonoidHom₁ and AddMonoidHom₁

```
structure MonoidHom1 (G H : Type) [Monoid G] [Monoid H] where
  toFun      : G -> H
  map_one    : toFun 1 = 1
  map_mul    : forall g g' : G, toFun (g * g') = toFun g * toFun g'

structure AddMonoidHom1 (G H : Type) [AddMonoid G] [AddMonoid H]
  where
  toFun      : G -> H
  map_zero   : toFun 0 = 0
  map_add    : forall g g' : G, toFun (g + g') = toFun g + toFun g'
```

Each bundles the underlying function (`toFun`) with a proof that identity maps to identity, and a proof that the operation is preserved.

RingHom₁ --- extends both

```
structure RingHom1 (R S : Type) [Ring R] [Ring S]
  extends MonoidHom1 R S, AddMonoidHom1 R S
```

A ring has two operators, so a ring homomorphism is exactly the conjunction of a monoid homomorphism (for $*$, giving $1 \mapsto 1$ and $f(ab) = f(a)f(b)$) and an additive-monoid homomorphism (for $+$, giving $0 \mapsto 0$ and $f(a + b) = f(a) + f(b)$) — no new fields are added.

6.2 Homomorphism classes

A `structure` like `MonoidHom` is one concrete *bundled* type of maps. A `class` like `MonoidHomClass` instead says: “any type `F` of maps $M \rightarrow N$ satisfying these laws can be treated as a monoid homomorphism,” which lets multiple unrelated wrapper types share the same lemma library.

```
class MonoidHomClass1 (F : Type) (M N : Type) [Monoid M] [Monoid N]
  where
  toFun      : F -> M -> N
  map_one    : forall f : F, toFun f 1 = 1
  map_mul    : forall f g g' : F, toFun f (g * g') = toFun f g * toFun f g'

instance (M N : Type) [Monoid M] [Monoid N] :
  MonoidHomClass2 (MonoidHom1 M N) M N where
  toFun      := MonoidHom1.toFun
  map_one    := fun f => f.map_one
  map_mul    := fun f => f.map_mul

instance (R S : Type) [Ring R] [Ring S] :
  MonoidHomClass2 (RingHom1 R S) R S where
  toFun      := fun f => f.toMonoidHom1.toFun
  map_one    := fun f => f.toMonoidHom1.map_one
  map_mul    := fun f => f.toMonoidHom1.map_mul
```

Since `RingHom` already extends `MonoidHom`, the `RingHom` instance reaches its multiplicative-monoid proof obligations via `toMonoidHom1` rather than re-proving them.

Key Idea

Structure vs. class, in one line: a `structure` homomorphism is a specific bundled object; a homomorphism `class` is a predicate over *any* carrier type `F`, letting new bundled types (e.g.

order-preserving homs, below) plug into existing generic lemmas instead of duplicating them.

6.3 Order-preserving homomorphisms

```

structure OrderPresHom (a b : Type) [LE a] [LE b] where
  toFun      : a -> b
  le_of_le   : forall x x' : a, x <= x' -> toFun x <= toFun x'

structure OrderPresMonoidHom (M N : Type) [Monoid M] [LE M] [Monoid N] [
  LE N]
  extends MonoidHom1 M N, OrderPresHom M N

class OrderPresHomClass (F : Type) (a b : outParam Type) [LE a] [LE b]

instance (a b : Type) [LE a] [LE b] :
  OrderPresHomClass (OrderPresHom a b) a b where

instance (a b : Type) [LE a] [Monoid a] [LE b] [Monoid b] :
  OrderPresHomClass (OrderPresMonoidHom a b) a b where

instance (a b : Type) [LE a] [Monoid a] [LE b] [Monoid b] :
  MonoidHomClass3 (OrderPresMonoidHom a b) a b where
  coe          := fun f => f.toOrderPresHom.toFun
  coe_injective' _ _ := OrderPresMonoidHom.ext
  map_one      := fun f => f.toMonoidHom1.map_one
  map_mul      := fun f => f.toMonoidHom1.map_mul

```

`OrderPresMonoidHom` extends *both* `MonoidHom` (operator-preserving) and `OrderPresHom` (order-preserving, $x \leq x' \Rightarrow f(x) \leq f(x')$), and instances register it under *both* `OrderPresHomClass` and `MonoidHomClass`: one wrapper type, two independent class memberships, each reusing an existing lemma library.

7 Full Hierarchy Reference

Class / Structure	Extends	Adds
<code>Dia</code>	—	operator $\diamond : \alpha \rightarrow \alpha \rightarrow \alpha$
<code>One</code>	—	distinguished element 1
<code>Inv</code>	—	function $(\cdot)^{-1} : \alpha \rightarrow \alpha$
<code>Semigroup</code>	<code>Dia</code>	associativity of $\diamond$
<code>DiaOneClass</code>	<code>Dia</code> , <code>One</code>	$\mathbf{1} \diamond a = a = a \diamond \mathbf{1}$
<code>Monoid</code>	<code>Semigroup</code> , <code>DiaOneClass</code>	(combination only)
<code>Group</code>	<code>Monoid</code> , <code>Inv</code>	$a^{-1} \diamond a = \mathbf{1}$
<code>AddSemigroup</code> / <code>Semigroup</code>	<code>Add</code> / <code>Mul</code>	associativity of $+$ / $*$
<code>AddMonoid</code> / <code>Monoid</code>	<code>AddSemigroup</code> + <code>AddZeroClass</code> (resp. <code>Mul</code> analogue)	(combination only)
<code>AddGroup</code> / <code>Group</code>	<code>AddMonoid</code> + <code>Neg</code> (resp. <code>Inv</code> analogue)	$-a + a = 0$ / $a^{-1} * a = 1$

Class / Structure	Extends	Adds
AddCommSemigroup / CommSemigroup	AddSemigroup Semigroup	/ commutativity
AddCommMonoid / CommMonoid	AddMonoid+AddCommSemigroup (resp. analogue)	(combination only)
AddCommGroup	AddGroup, AddCommMonoid	(combination only; = Abelian group)
Ring	AddGroup, Monoid, MulZeroClass	left/right distributivity
AddCommGroup (for a Ring)	(instance, not a hypothesis)	add_comm derived from distributivity
SMul	—	action $\bullet : R \rightarrow M \rightarrow M$
Module	SMul over Ring R and AddCommGroup M	zero_smul, one_smul, mul_smul, add_smul, smul.add
selfModule	(instance) Module $R R$	scalar-mul instantiated to $*$
MonoidHom	—	toFun, $f(1)=1$, $f(gg')=f(g)f(g')$
AddMonoidHom	—	toFun, $f(0)=0$, $f(g+g')=f(g)+f(g')$
RingHom	MonoidHom, AddMonoidHom	(combination only)
MonoidHomClass	—	predicate: type F acts like a MonoidHom on $M \rightarrow N$
OrderPresHom	—	toFun, $x \leq x' \Rightarrow f(x) \leq f(x')$
OrderPresMonoidHom	MonoidHom, OrderPresHom	(combination only)
OrderPresHomClass	—	predicate: type F acts like an OrderPresHom